



Duka POS — Launch Pack

Launch · Demos · Hosting · Integrations · Cloud Sync

The owner's step-by-step guide (no technical knowledge needed)

July 2026

Duka POS — Launch, Demo & Hosting Guide

For the owner (non-technical). This is the "what do I actually do now" document. Follow it top to bottom and you're live.

The 3 pieces of your product

Piece	What it is	Where it lives
The POS app	What your clients use every day	Vercel (free)
The backend	Auto-billing, WhatsApp/SMS sending, M-PESA, Airtel, eTIMS, Duka AI	Render (~\$7/mo)
The website	Your marketing page that sells the POS	Vercel (free)

Plus your **Super-Admin portal** (see every shop, charge, suspend) at `https://YOUR-BACKEND.onrender.com/admin`, and **cloud sync** on Supabase (free — see `CLOUD-SYNC.md`).

Recommended hosting: Vercel + Render + Supabase. This combo is the best fit for Duka: Vercel is the best free host for the app and website (instant, global, free SSL), Render runs the backend 24/7 for the auto-billing scheduler, and Supabase gives you the database + login system free. Don't pay for anything else until you have ~50 shops.

Part 1 — Put everything online (one afternoon)

A. The POS app on Vercel (10 min)

- Go to <https://vercel.com> → sign up with your GitHub account.
- Add New → Project** → import **brianzeey/duka-pos** → Deploy. (All settings are auto-detected — just click Deploy.)
- You get `https://duka-pos-xxxx.vercel.app`. In **Settings → Domains** you can rename it to `duka-pos.vercel.app` if free, or attach a domain you buy (e.g. `dukapos.co.ke`, ~KES 1,000/yr).

B. The backend on Render (15 min)

- Go to <https://render.com> → sign up with GitHub.
- New → Web Service** → choose **duka-pos** → set **Root Directory** to `server` → the included `render.yaml` fills in the rest → Create.
- In **Environment**, add at minimum:
 - `ADMIN_TOKEN` = a long secret password (this is your master key)
 - `DATABASE_URL` = your Supabase connection string (Supabase dashboard → Project Settings → Database → Connection string → URI)
- Copy your backend URL, e.g. `https://duka-pos-api.onrender.com`.
- Back on Vercel: **Settings → Environment Variables** → `VITE_API_URL` = that URL → Redeploy.

C. Cloud sync on Supabase (10 min)

Follow `CLOUD-SYNC.md` (run one script, paste two values into Vercel).

D. The marketing website on Vercel (5 min)

1. Vercel → **Add New** → **Project** → import **duka-pos** again.
2. This time set **Root Directory** to `website` and Framework to **Other**.
3. Deploy → you get your sales page, e.g. `https://dukapos.vercel.app`.
4. Point its "Open the live demo" button at your app URL (it's one link in `website/index.html` — ask any developer, or me, to change it in seconds).

E. Go live with money (whenever ready)

Follow `INTEGRATIONS.md` to switch M-PESA, Airtel, eTIMS and Duka AI from simulation to live — each is just pasting credentials into Render.

Part 2 — How to demo Duka POS (your sales weapon)

The fastest demo: your own phone. The app ships with demo data — a stocked duka, sales history, and debtors — so the demo is impressive with zero preparation.

The 5-minute script that sells:

1. **Open the app on your phone** (or the client's!) — go to your Vercel URL, "Add to Home Screen". *"It's an app, no installation, works on the phone you already have."*
2. **Make a sale in 5 seconds.** Tap two products → Take payment → Exact cash. *"Your cashier learns this in one minute."*
3. **The killer feature — mkopo.** Sell on Credit → pick a customer → open **Debts**. Show the balance, then tap **Remind** → **WhatsApp**. Their jaw drops when the message appears with the till number already inside. *"This app collects your debts for you."*
4. **Show Duka AI.** Ask "Who owes me money?" then "What should I restock?" *"It's like having an accountant inside the till."*
5. **Show Reports** (today's money, best sellers) and **Settings** → **Staff** (cashier PINs can't see your profit).
6. **Turn on airplane mode and sell again.** *"No bundles? No problem."*
7. Close: *"14 days free, from KES 1,900 a month, cancel anytime. Tuanze?"*

Demo hygiene: after each demo, Settings → **Reset demo data** so the next demo starts clean. For remote demos, just send the app link on WhatsApp — it works instantly on their phone with the same demo data.

Part 3 — Launch checklist

Before your first paying client:

- ☐ App + backend + website deployed (Part 1 A, B, D)
- ☐ `ADMIN_TOKEN` set; open `/admin` and confirm you can see shops
- ☐ Cloud sync set up (Part 1 C)
- ☐ Your WhatsApp number is on the website's contact button
- ☐ Print `docs/Duka-POS-Owner-Guide.pdf` for yourself — it's your manual

Onboarding each new shop (15 minutes, in person or on a call):

1. Open your app URL on their device → Add to Home Screen.
2. Settings → their shop name, phone, and **their M-PESA till/Paybill** (this is what goes into their debt reminders).

- 3. Settings → Staff → set the owner PIN, add cashiers.
- 4. Add their top 20 products together; teach them to add the rest.
- 5. Billing → pick the right plan for their size. Their 14-day trial starts.
- 6. (Optional) Settings → Cloud sync → create their account for multi-device.
- 7. Show them one credit sale + one WhatsApp reminder. That's the hook.

Getting paid every month (this is automated):

- Trial ends → the app reminds them → they tap Pay → M-PESA STK on their phone → done. If they set auto-pay, the server charges them automatically.
- Don't pay → grace (7 days, warnings) → the POS **holds** (day 8-14) → suspended (day 15+). They call you fast — the POS matters to them by then.
- You watch everything (and can charge/extend anyone) from `/admin`.

Suggested launch sequence:

- 1. Week 1: onboard your 2-3 friendliest waiting clients personally. Sit in their shop for an hour. Fix friction. Get testimonials.
- 2. Week 2: onboard the rest of the waiting list; ask each happy client to name one other shop owner ("mtaje mmoja" — referrals are everything).
- 3. Then: print simple flyers with the website + your WhatsApp, walk the business streets, demo on the spot from your phone. Every demo ends with installing it on THEIR phone with the trial running — the app sells itself for 14 days.

Costs summary

Item	Cost
Vercel (app + website)	Free
Render backend	~\$7/month (~KES 900)
Supabase (database + sync + login)	Free tier
SMS via Africa's Talking	~KES 0.8/SMS, only if you enable SMS
WhatsApp Cloud API	Free tier ~1,000 conversations/month
Claude API for Duka AI	A few dollars/month, optional
Domain (optional)	~KES 1,000/year

One Standard-plan client (KES 4,900/mo) covers your entire monthly bill several times over.

Duka POS — Integrations Guide (M-PESA, Airtel Money, KRA eTIMS)

Written for a non-technical owner. Every integration below is already built into Duka POS. Out of the box they run in **simulation mode** — the app behaves exactly as if they were live, so you can demo and sell today. Going live is just "get credentials → paste them into your server settings → restart".

Where do the credentials go? On Render (your backend host), open your service → **Environment** → add the variables named below → **Save** (it restarts automatically). That's the whole process, for all three.

1) M-PESA — Daraja (Safaricom)

What it does in Duka:

- Customers' M-PESA payments at the till (you type in the M-PESA code).
- Your shops' **subscription payments to YOU** via STK push — the shop owner taps "Pay", their phone pops up the M-PESA PIN prompt, done.
- **Auto-billing:** the server automatically sends the STK push when a shop's month is up, and Paybill payments are auto-detected and matched to the shop.

Get credentials (about 30-60 min, one time):

1. Go to <https://developer.safaricom.co.ke> and create an account.
2. Click **Add App**, tick **Lipa na M-PESA Sandbox**, and note the **Consumer Key** and **Consumer Secret**.
3. To go to production: click **Go Live** in the portal. You'll need your Paybill or Till number (get one from Safaricom/your bank if you don't have one yet — "Lipa na M-PESA Buy Goods" from any Safaricom shop).
4. Safaricom emails you the production keys + **Passkey**.

Paste into Render → Environment:

```
MPESA_ENV=production           (or sandbox while testing)
MPESA_CONSUMER_KEY=...
MPESA_CONSUMER_SECRET=...
MPESA_SHORTCODE=...           (your Paybill or Till number)
MPESA_PASSKEY=...
MPESA_TX_TYPE=CustomerPayBillOnline (or CustomerBuyGoodsOnline for a Till)
```

Done. STK pushes are real, the auto-biller charges real money, and Paybill payments renew shops automatically.

2) Airtel Money

What it does in Duka:

- An **Airtel** button at the till so cashiers can record Airtel Money sales.
- Your Airtel number is included in **debt reminders and receipts** so customers know how to pay you back.
- Subscription payments via Airtel Money push (the customer approves on their phone, like an STK push).

Get credentials (about 30 min, one time):

1. Go to <https://developers.airtel.africa> and sign up (free).
2. Click **My Apps** → **Create App**, tick the **Collection** product.
3. Copy the **Client ID** and **Client Secret** shown on the app page.
4. Ask Airtel (through the portal) to approve you for production when ready.

Paste into Render → Environment:

```
AIRTEL_ENV=production          (or staging while testing)
AIRTEL_CLIENT_ID=...
AIRTEL_CLIENT_SECRET=...
```

In the app: Settings → "How customers pay you" → enter your **Airtel Money number** — it then appears in reminders and on receipts, even with no backend at all.

3) KRA eTIMS (electronic tax invoices)

What it does in Duka:

- Prints your **KRA PIN** on every receipt (works instantly, no setup).
- With credentials, every sale is also **submitted to KRA automatically** in the background — the cashier does nothing extra.

Turn on the easy part right now (no credentials needed): In the app → **Settings** → **Tax & KRA eTIMS** → switch on eTIMS → type the shop's KRA PIN. Receipts now carry the PIN.

Go fully live (one time, per shop):

1. The shop registers for eTIMS on <https://etims.kra.go.ke> (eCitizen login). Choose the **eTIMS Lite / OSCU (online)** option for small shops.
2. KRA approves and issues the technical details: an **API endpoint**, the shop's **PIN (TIN)**, a **branch id** (usually `00`) and a **device key**. A KRA-listed integrator can do this step for you — it's a standard service.
3. Paste into Render → Environment:

```
ETIMS_ENDPOINT=...           (KRA gives you this URL)
ETIMS_TIN=P0XXXXXXXXXX
ETIMS_BHF_ID=00
ETIMS_CMC_KEY=...
```

Until those are set, eTIMS submissions are simulated (and logged), so the till flow is identical either way.

4) Duka AI (bonus — the in-app assistant)

What it does: every shop gets a **Duka AI** tab — they ask questions in plain language ("Who owes me money?", "What should I restock?") and get answers from their own sales, debts and stock.

- **No setup:** it already works — answers are computed on the device.
- **Smarter answers with Claude:** get an API key at <https://console.anthropic.com> → API Keys, then add on Render:

```
ANTHROPIC_API_KEY=sk-ant-...
```

Quick status check

Open `https://YOUR-BACKEND.onrender.com/api/health` in a browser. You'll see each integration listed as `live` or `simulation` — that's how you confirm a credential was picked up.

Duka POS — Cloud Sync (multi-device) Setup

Cloud sync makes Duka a true online POS: a shop signs in once on each device (email + password) and its products, customers, sales and debts stay in step on all of them, live. Offline still works — the device keeps selling and syncs when it's back online.

It runs on **Supabase** (a hosted database with a generous free tier). Setup is one-time, about 10 minutes, and covers ALL your shops.

Step 1 — Open your Supabase project

Go to <https://supabase.com/dashboard> and open your project (you already have one named **duka-pos** — use that one).

Step 2 — Run the setup script (once)

1. In the left menu click **SQL Editor** → **New query**.
2. Open the file `supabase/schema.sql` from this repository, copy ALL of it, paste it into the editor, and click **Run**.
3. You should see "Success. No rows returned". That's it — the database now has the `shop_state` table, security rules (each shop can only ever see its own data), and live updates enabled.

Step 3 — Tell the app about your project

1. In Supabase, click **Project Settings** (gear icon) → **API**.
2. Copy two values: the **Project URL** and the **anon public** key.
3. In Vercel (where the app is hosted): your project → **Settings** → **Environment Variables** → add both:

```
VITE_SUPABASE_URL=https://xxxx.supabase.co  
VITE_SUPABASE_ANON_KEY=eyJ...
```

4. Redeploy the app (Vercel → Deployments → ... → Redeploy).

Step 4 — Each shop signs in (they do this themselves)

In the app: **Settings** → **Cloud sync** → the shop creates an account with an email + password (or signs in on a second device with the same one). The status turns **Live** — from then on every device signed into that account shares the same data in real time.

How conflicts are handled (so you can reassure clients)

- **Sales are never lost.** If two devices sell offline at the same time, both sales appear once they're back online.
- Debt payments recorded on any device are kept.
- Receipt numbers never collide backwards.

Troubleshooting

- **Status says "off"** → the two `VITE_SUPABASE_*` variables aren't set on Vercel (or you didn't redeploy after adding them).
- **Sign-up email confirmation annoying?** In Supabase: **Authentication** → **Providers** → **Email** → **turn OFF "Confirm email"** — shops can then sign in immediately.
- **Status says "error"** → run Step 2 again; the script is safe to re-run.